

September 16, 2026 at 22:31

1. Introduction. The van der Waerden number $W(3, k)$ is the smallest n such that every string $x_1 \dots x_n$ of 0s and 1s has three equally spaced 0s or k equally spaced 1s; the clauses $waerden(3, k; n)$ are satisfiable exactly when $n < W(3, k)$. Exercise 7.2.2–6, on page 317, asks:

6. [HM37] Use the Local Lemma to show that $W(3, k) = \Omega(k^2/(\log k)^3)$.

The answer on page 549 reads, in full:

Let each x_i be 0 with probability $p = (2 \ln k)/k$, and let n be at most $k^2/(\ln k)^3$. There are two kinds of “bad events”: A_i , a set of three equally spaced 0s, occurs with probability $P = p^3$; and A'_j , a set of k equally spaced 1s, occurs with probability $P' = (1 - p)^k \leq \exp(-kp) = 1/k^2$. In the lopsidedependency graph, which is bipartite, each A_i is adjacent to at most $D = 3k^3/((k - 1)(\ln k)^3)$ nodes A'_j ; each A'_j is adjacent to at most $d = \frac{3}{2}k^3/(\ln k)^3$ nodes A_i . By Theorem L, we want to show that, for all sufficiently large values of k , $P \leq y(1 - x)^D$ and $P' \leq x(1 - y)^d$, for some x and y .

Choose x and y so that $(1 - x)^D = 1/2$ and $y = 2P$. Then $x = \Theta((\log k)^3/k^2)$ and $y = \Theta((\log k)^3/k^3)$; hence $(1 - y)^d = \exp(-yd + O(y^2d)) = O(1)$. [See T. Brown, B. M. Landman, and A. Robertson, *J. Combinatorial Theory* **A115** (2008), 1304–1309.]

2. The answer is a sketch, and a sketch of this kind hides constants. This program brings them out.

1. *Events.* The probabilities P and P' , and the claim that the lopsidedependency graph is bipartite, are checked exactly on short strings: condition (133) of Lemma L by enumeration of all strings, and the rule of answer 351 for when two events must be adjacent.
2. *Degrees.* The largest numbers of neighbors are computed and compared with D and d .
3. *Numbers.* With the answer’s choices, yd is exactly 24, so $(1 - y)^d$ tends to e^{-24} . The second inequality then holds only when $(\ln k)^3 \geq 3e^{24}/\ln 2$, that is, when $\ln k$ exceeds about 4858. The program computes that threshold in logarithms, and also shows that the same argument with $p = (3 \ln k)/k$ gives $W(3, k) > k^2/(81(\ln k)^2)$ for every $k \geq 8$.
4. *Concrete bounds.* For each k up to 30, the largest n for which Theorem L can be applied with the true degrees, compared with the known values of $W(3, k)$; and, for k up to 10^8 , the largest n allowed by the answer’s own D and d with free constants, compared with $k^2/(\ln k)^3$.
5. *Resampling.* Algorithm M finds strings with no bad event, and Theorem M bounds how often it resamples; both are checked.
6. *The solver.* The `sat` package recomputes $W(3, k)$ for $k \leq 10$, and a plain backtracking search confirms the smallest values.

I wrote this on 16 September 2026.

3. Every claim is a mode.

```

package main
import (
    "context"
    "flag"
    "fmt"
    "math"
    "math/rand/v2"
    "os"
    "github.com/sjnam/sat"
)
⟨Types 9⟩
⟨Global variables 5⟩
⟨Functions 6⟩
func main() {
    ⟨Read the command line 4⟩
    if on("events") {
        ⟨Check the events and the lopsidedependency graph 12⟩
    }
    if on("degrees") {
        ⟨Check the degree bounds  $D$  and  $d$  21⟩
    }
    if on("numbers") {
        ⟨Check the inequalities of Theorem L numerically 24⟩
    }
    if on("concrete") {
        ⟨Compute the bounds that Theorem L gives for concrete  $k$  28⟩
    }
    if on("resample") {
        ⟨Run Algorithm M 30⟩
    }
    if on("solver") {
        ⟨Recompute small values of  $W(3,k)$  35⟩
    }
    ⟨Say how it went 7⟩
}

```

4. $\langle \text{Read the command line 4} \rangle \equiv$

```

flag.StringVar(&mode, "mode", "all",
    "events, degrees, numbers, concrete, resample, solver, or all")
flag.IntVar(&reps, "reps", 1000, "how many runs of Algorithm M")
flag.IntVar(&wmax, "wmax", 10, "the largest k for which  $W(3,k)$  is recomputed")
flag.Uint64Var(&seed, "seed", 20260916, "seed for the random choices")
flag.Parse()

```

This code is used in section 3.

5. $\langle$ Global variables 5 $\rangle \equiv$

```

var (
  mode string
  reps, wmax int
  seed uint64
  failures int
)

```

This code is also defined in section 36.

This code is used in section 3.

6. Each claim is announced, checked, and marked; one **FAIL** anywhere makes the program exit with a nonzero status.

 $\langle$ Functions 6 $\rangle \equiv$

```

func on(m string) bool { return mode  $\equiv$  m  $\vee$  mode  $\equiv$  "all" }
func claim(ok bool, format string, args ... any) {
  mark := "ok_␣_␣"
  if  $\neg$ ok {
    mark = "FAIL"
    failures ++
  }
  fmt.Printf("_␣_␣%s_␣_␣%s\n", mark, fmt.Sprintf(format, args...))
}

```

This code is also defined in sections 8, 10, 11, 16, 18, 19, 20, 22, 27, 32, 34, 39.

This code is used in section 3.

7. $\langle$ Say how it went 7 $\rangle \equiv$

```

if failures > 0 {
  fmt.Printf("_␣_␣%d_␣_␣claims_␣_␣failed.\n", failures)
  os.Exit(1)
}
fmt.Println("_␣_␣Every_␣_␣claim_␣_␣checked_␣_␣out.")

```

This code is used in section 3.

8. Progressions and events. Positions are $1, \dots, n$. *aps* lists the arithmetic progressions of length L inside them, by increasing difference.

⟨ Functions 6 ⟩ +≡

```

func aps(n, L int) [[]]int {
  var out [[]]int
  for d := 1; (L - 1) * d ≤ n - 1; d++ {
    for a := 1; a + (L - 1) * d ≤ n; a++ {
      ap := make([]int, L)
      for t := range ap {
        ap[t] = a + t * d
      }
      out = append(out, ap)
    }
  }
  return out
}

```

9. An event is a progression and a bit: the three-term progressions carry 0, for the events A_i , and the k -term ones carry 1, for the events A'_j .

⟨ Types 9 ⟩ ≡

```

type event struct {
  pos []int
  one bool
}

```

This code is used in section 3.

10. ⟨ Functions 6 ⟩ +≡

```

func events(k, n int) []event {
  var evs []event
  for _, ap := range aps(n, 3) {
    evs = append(evs, event{ap, false})
  }
  for _, ap := range aps(n, k) {
    evs = append(evs, event{ap, true})
  }
  return evs
}

```

11. A string of at most 62 bits is a bit vector, bit i for x_i .

⟨ Functions 6 ⟩ +≡

```

func happens(e event, x uint64) bool {
  for _, i := range e.pos {
    if (x >> i & 1 ≡ 1) ≠ e.one {
      return false
    }
  }
  return true
}

```

12. Events. With $x_i = 0$ with probability p , independently, an event A_i has probability p^3 and an event A'_j has probability $(1-p)^k$; and $(1-p)^k \leq e^{-kp}$ because $1-p \leq e^{-p}$, with $e^{-kp} = 1/k^2$ when $p = (2 \ln k)/k$.

The claim that needs care is the graph. Lemma L on page 266 asks, in (133), that $\Pr(A_i \mid \bar{A}_{j_1} \cap \dots \cap \bar{A}_{j_t}) \leq p_i$ whenever none of $j_1, \dots, j_t$ is a neighbor of i . The answer's graph joins A_i only to the A'_j that share a position with it, so events of the same kind are never neighbors even when they overlap. That is where “lopsided” comes in: three 0s are only made less likely by knowing that other triples are not all 0.

The program checks (133) by enumerating all 2^n strings with their exact probabilities, for $n \leq 10$, $k = 3$ and 4, and three values of p . For every event it conditions on all its non-neighbors at once, and on 30 random subsets of them. There are at most 40 events, so the events that happen in a string fit in one word, and a set of events is a word too.

$\langle$ Check the events and the lopsidedness graph 12 $\rangle \equiv$

```
fmt.Println("6. the events")
 $\langle$  Check the probabilities of the events 13  $\rangle$ 
r := rand.New(rand.NewPCG(seed, 6))
var nCond, badCond int
for _, k := range []int { 3, 4 } {
    for n := k; n ≤ 10; n++ {
        for _, p := range []float64 { 0.2, 0.5, 0.8 } {
             $\langle$  Check condition (133) for all events on n positions 14  $\rangle$ 
        }
    }
}
claim(badCond == 0, "%d conditional probabilities on up to 10 positions: " +
    "none exceeds Pr(A)", nCond)
 $\langle$  Check the rule of answer 351 17  $\rangle$ 
```

This code is used in section 3.

13. $\langle$ Check the probabilities of the events 13 $\rangle \equiv$

```
badP := 0
for k := 2; k ≤ 1000000; k = k * 3/2 + 1 {
    p := 2 * math.Log(float64(k))/float64(k)
    Pp := math.Pow(1-p, float64(k))
    if p ≥ 1 ∨ Pp > math.Exp(-float64(k) * p) * (1 + 1e-12) ∨
        math.Abs(math.Exp(-float64(k) * p) * float64(k * k) - 1) > 1e-9 {
        badP++
    }
}
claim(badP == 0, "(1-p)^k ≤ exp(-kp) = 1/k^2 when p = 2 ln k/k, for k" +
    "from 2 to a million")
```

This code is used in section 12.

14. The weight of a string is $\prod p^{[x_i=0]}(1-p)^{[x_i=1]}$. Two events are neighbors when one is an A and the other an A' and they share a position.

⟨ Check condition (133) for all events on n positions 14 ⟩ $\equiv$

```

evs := events(k, n)
w, hap := make([]float64, 1 << n), make([]uint64, 1 << n)
for x := uint64(0); x < 1 << n; x++ {
  w[x] = 1
  for i := 1; i ≤ n; i++ {
    if x << 1 >> i & 1 ≡ 1 {
      w[x] *= 1 - p
    } else {
      w[x] *= p
    }
  }
}
for b, e := range evs {
  if happens(e, x << 1) {
    hap[x] |= 1 << b
  }
}
}
for a, ea := range evs {
  var others uint64
  for b, eb := range evs {
    if b ≠ a ∧ (ea.one ≡ eb.one ∨ ¬share(ea, eb)) {
      others |= 1 << b
    }
  }
}
⟨ Compare the conditional probabilities of ea with its own 15 ⟩
}

```

This code is used in section 12.

15. $\langle$ Compare the conditional probabilities of ea with its own 15 $\rangle \equiv$

```

alone := math.Pow(p, 3)
if ea.one {
  alone = math.Pow(1 - p, float64(k))
}
for trial := 0; trial ≤ 30; trial++ {
  sub := others
  if trial > 0 {
    sub &= r.Uint64()
  }
  var both, cond float64
  for x := uint64(0); x < 1 << n; x++ {
    if hap[x] & sub == 0 {
      cond += w[x]
      if hap[x] >> a & 1 == 1 {
        both += w[x]
      }
    }
  }
  nCond++
  if cond > 0 ∧ both/cond > alone * (1 + 1e - 9) {
    badCond++
  }
}

```

This code is used in section 14.

16. $\langle$ Functions 6 $\rangle + \equiv$

```

func share(e, f event) bool {
  for _, i := range e.pos {
    for _, j := range f.pos {
      if i == j {
        return true
      }
    }
  }
  return false
}

```

17. Algorithm M needs more than (133). Answer 351, due to Moser and Tardos, says when events i and j must be joined: when some setting makes A_i false and A_j true while some change to the variables of A_j would make A_i true, or the same with i and j exchanged. The program applies that rule literally, over all settings and all changes, for every ordered pair, and compares it with the answer's bipartite graph; since that graph is symmetric, one direction of the rule already has to match it.

⟨ Check the rule of answer 351 17 ⟩ $\equiv$

```

var pairs, badRule int
for _, k := range []int { 3, 4 } {
  for n  $:= k$ ;  $n \leq 9$ ;  $n++$  {
    evs  $:=$  events(k, n)
    for a  $:=$  range evs {
      for b  $:=$  range evs {
        if  $a \equiv b$  {
          continue
        }
        pairs++
        want  $:=$  evs[a].one  $\neq$  evs[b].one  $\wedge$  share(evs[a], evs[b])
        if mustJoin(evs[a], evs[b], n)  $\neq$  want {
          badRule++
        }
      }
    }
  }
}

claim(badRule  $\equiv$  0, "%d ordered pairs of events on up to 9 positions: the rule" +
  "of answer 351 gives exactly the bipartite graph", pairs)

```

This code is used in section 12.

18. Is there a setting with A_i false and A_j true, and a change of the variables of A_j that makes A_i true?

⟨ Functions 6 ⟩ $+=$

```

func mustJoin(ei, ej event, n int) bool {
  for x  $:=$  uint64(0);  $x < 1 \ll n$ ;  $x++$  {
    if happens(ei, x  $\ll$  1)  $\vee$   $\neg$ happens(ej, x  $\ll$  1) {
      continue
    }
    for c  $:=$  1;  $c < 1 \ll \text{len}(ej.pos)$ ;  $c++$  {
      y  $:=$  x  $\ll$  1
      for t, i  $:=$  range ej.pos {
        if  $c \gg t \ \& \ 1 \equiv 1$  {
          y  $\oplus=$  1  $\ll$  i
        }
      }
      if happens(ei, y) {
        return true
      }
    }
  }
  return false
}

```

19. Degrees. An event A_i has three positions, and each position lies in at most $k\lfloor(n-1)/(k-1)\rfloor$ progressions of length k : it can be term 1, $\dots$, k , and the difference is at most $(n-1)/(k-1)$. So A_i has at most $3k(n-1)/(k-1)$ neighbors, which is below the answer's D when $n \leq k^2/(\ln k)^3$. An event A'_j has k positions, and a position x lies in at most $\lfloor(n-x)/2\rfloor + \min(x-1, n-x) + \lfloor(x-1)/2\rfloor \leq n-1$ progressions of length 3. So A'_j has at most $k(n-1)$ neighbors, below the answer's $d = 3kn/2$.

degrees counts the distinct neighbors exactly and returns the largest numbers, for the A 's and for the A' 's.

(Functions 6) $\equiv$

```

func degrees(k, n int)(int, int) {
  three, long := aps(n, 3), aps(n, k)
  in3, inK := make([[]]int, n + 1), make([[]]int, n + 1)
  for i, ap := range three {
    for _, x := range ap {
      in3[x] = append(in3[x], i)
    }
  }
  for j, ap := range long {
    for _, x := range ap {
      inK[x] = append(inK[x], j)
    }
  }
  return most(three, inK, len(long)), most(long, in3, len(three))
}

```

20. The most distinct members of *through* met by one progression of *from*.

(Functions 6) $\equiv$

```

func most(from [[]]int, through [[]]int, size int) int {
  best := 0
  seen := make([int], size)
  for i, ap := range from {
    c := 0
    for _, x := range ap {
      for _, j := range through[x] {
        if seen[j]  $\neq$  i + 1 {
          seen[j], c = i + 1, c + 1
        }
      }
    }
    best = max(best, c)
  }
  return best
}

```

21. $\langle \text{Check the degree bounds } D \text{ and } d \text{ 21} \rangle \equiv$

```

fmt.Println("6. the degrees")
var nd, badD, badd int
worstD, worstd := 0.0, 0.0
for k := 3; k ≤ 12; k++ {
  for n := k; n ≤ 100; n++ {
    D, d := degrees(k, n)
    nd++
    bD := 3 * float64(k) * float64(n-1) / float64(k-1)
    bd := float64(k) * float64(n-1)
    if float64(D) > bD + 1e-9 ∨ float64(D) > 3 * float64(k * n) / float64(k-1) {
      badD++
    }
    if float64(d) > bd ∨ float64(d) > 1.5 * float64(k * n) {
      badd++
    }
    worstD, worstd = max(worstD, float64(D)/bD), max(worstd, float64(d)/bd)
  }
}
claim(badD == 0, "%d pairs (k,n): A has at most 3k(n-1)/(k-1) ≤ D neighbors; the largest ratio is %.3f", nd, worstD)
claim(badd == 0, "A' has at most k(n-1) ≤ d neighbors; the largest ratio to k(n-1) is %.3f", worstd)

```

This code is used in section 3.

22. Numbers. Write $L = \ln k$. The answer's choices make $y = 2p^3 = 16L^3/k^3$ and $d = \frac{3}{2}k^3/L^3$, so $yd = 24$ exactly, and $(1 - y)^d = e^{-24}(1 + O(y))$. That is a constant, which is what the argument needs; but the answer writes “ $= O(1)$ ”, an upper bound, when the step needs a lower bound. And the constant is small. Since $x \approx \ln 2/D \approx (\ln 2)L^3/(3k^2)$ and $P' \approx 1/k^2$, the inequality $P' \leq x(1 - y)^d$ becomes

$$1 \leq \frac{\ln 2}{3} L^3 e^{-24} (1 + o(1)), \quad \text{or} \quad L \geq \exp((24 + \ln 3 - \ln \ln 2)/3) \approx 4858.$$

So “sufficiently large” means $k > e^{4858}$, a number of 2110 digits.

margin evaluates $\ln(x(1 - y)^d) - \ln P'$ for $p = cL/k$ and $n = ak^2/L^3$, with D and d exactly as in the answer and x and y chosen as there. Everything is done in logarithms, because k itself does not fit in a floating-point number.

⟨ Functions 6 ⟩ +≡

```
func margin(L, c, a float64) float64 {
  lnp := math.Log(c) + math.Log(L) - L
  p := math.Exp(lnp)
  lnn := math.Log(a) + 2 * L - 3 * math.Log(L)
  ⟨ Compute  $\ln P'$ ,  $\ln x$ , and  $\ln(1 - y)^d$  23 ⟩
  return lnx + lnpow - lnPp
}
```

23. For large k the terms p , y , and $1/k$ underflow, so the series $\ln(1 - t) = -t(1 + t/2 + \dots)$ takes over where they are negligible.

⟨ Compute $\ln P'$, $\ln x$, and $\ln(1 - y)^d$ 23 ⟩ ≡

```
lnPp := -c * L * (1 + p/2)
if L < 30 {
  lnPp = math.Exp(L) * math.Log1p(-p)
}
lnD := math.Log(3) + lnn - math.Log1p(-math.Exp(-L))
lnd := math.Log(1.5) + L + lnn
lnt := math.Log(math.Ln2) - lnD
lnx := lnt
if lnt > -700 {
  lnx = math.Log(-math.Expm1(-math.Exp(lnt)))
}
lny := math.Ln2 + 3 * lnp
y := math.Exp(lny)
lnpow := -math.Exp(lnd + lny) * (1 + y/2)
if y > 1e-8 {
  lnpow = math.Exp(lnd) * math.Log1p(-y)
}
```

This code is used in section 22.

24. The program checks $yd = 24$, finds the threshold by bisection, compares it with the closed form, and checks that the margin changes sign only there, on a logarithmic grid of L from 1 to 10^6 .

⟨ Check the inequalities of Theorem L numerically 24 ⟩ $\equiv$

```

fmt.Println("6. the inequalities of Theorem L")
badYD := 0
for _, L := range []float64 { 2, 10, 100, 4858, 1e5 } {
    lnyd := math.Ln2 + 3 * (math.Log(2 * L) - L) + math.Log(1.5) + 3 * L - 3 * math.Log(L)
    if math.Abs(math.Exp(lnyd) - 24) > 1e-9 {
        badYD++
    }
}
claim(badYD == 0, "yd=24 exactly, so (1-y)^d tends to exp(-24)=%.3g",
    math.Exp(-24))
lo, hi := 1.0, 1e6
for i := 0; i < 200; i++ {
    if mid := (lo + hi)/2; margin(mid, 2, 1) ≥ 0 {
        hi = mid
    } else {
        lo = mid
    }
}
closed := math.Exp((24 + math.Log(3) - math.Log(math.Ln2))/3)
claim(math.Abs(hi - closed) < 1e-6 * closed, "the second inequality holds from " +
    "ln k=%.4f on, as the closed form %.4f predicts: k>10^%.1f", hi, closed,
    hi/math.Ln10)
⟨ Check that the margin changes sign once 25 ⟩
⟨ Check the variant with  $p = (3 \ln k)/k$  26 ⟩

```

This code is used in section 3.

25. ⟨ Check that the margin changes sign once 25 ⟩ $\equiv$

```

signs := 0
for i := 0; i ≤ 6000; i++ {
    L := math.Pow(10, 6 * float64(i)/6000)
    if (margin(L, 2, 1) ≥ 0) ≠ (L ≥ hi) {
        signs++
    }
}
claim(signs == 0, "on 6001 values of ln k from 1 to 10^6 the margin is " +
    "negative below the threshold and positive above it")

```

This code is used in section 24.

26. Nothing in the argument forces $p = (2 \ln k)/k$. With $p = (3 \ln k)/k$ we have $P' \leq 1/k^3$, and with $n = ak^2/L^3$ the product yd becomes $81a$; the second inequality then reads, up to lower-order terms, $81a + \ln a \leq L + 3 \ln L + \ln(\ln 2/3)$. Taking $a = L/81$, that is, $n = k^2/(81L^2)$, it holds for every L . So the same argument proves $W(3, k) = \Omega((k/\log k)^2)$, the bound published by Y. Li and J. Shu in *Advances in Applied Mathematics* **44** (2010), 243–247. The program checks the exact margin, on the same grid, from $k = 8$; below that $p = (3 \ln k)/k$ is so large that $y = 2p^3$ is not a probability, or nearly so, and the margin is negative at $k \approx 7.7$.

⟨ Check the variant with $p = (3 \ln k)/k$ [26](#) ⟩ $\equiv$

```

badVar := 0
for i := 0; i ≤ 6000; i++ {
  L := math.Pow(10, 6 * float64(i)/6000)
  if L < math.Log(8) {
    continue
  }
  if ¬(margin(L, 3, L/81) ≥ 0) {
    badVar++
  }
}
claim(badVar == 0, "with p = 3 ln k/k and n = k^2/(81 (ln k)^2) both +
  "inequalities hold for all k >= 8 on the grid: %d exceptions", badVar)

```

This code is used in section [24](#).

27. Concrete bounds. The answer's argument, like any Local Lemma argument, produces a definite bound for each k once the constants are free. *feasible* looks for p and x that satisfy Theorem L, with y as small as the first inequality allows, given the numbers D and d of neighbors; it searches logarithmic grids, so every success is a proof and a failure is only a failure to find one.

⟨ Functions 6 ⟩ +≡

```

func feasible( $k, D, d$  float64)(bool, float64, float64, float64) {
  for  $ip := 1; ip < 300; ip++$  {
     $p := \text{math.Pow}(10, -9 + 9 * \text{float64}(ip)/300)$ 
     $\ln Pp := k * \text{math.Log1p}(-p)$ 
    for  $ix := 1; ix < 300; ix++$  {
       $x := \text{math.Pow}(10, -16 + 16 * \text{float64}(ix)/300)$ 
       $\ln y := 3 * \text{math.Log}(p) - D * \text{math.Log1p}(-x)$ 
      if  $\ln y \geq 0$  {
        break
      }
       $y := \text{math.Exp}(\ln y)$ 
      if  $\ln Pp \leq \text{math.Log}(x) + d * \text{math.Log1p}(-y)$  {
        return true,  $p, x, y$ 
      }
    }
  }
  return false, 0, 0, 0
}

```

28. For $k \leq 30$ the degrees are the true ones, and the bound is compared with the table on page 189: $W(3, k)$ is known for $k \leq 19$, and at least the tabulated number for $20 \leq k \leq 30$.

⟨ Compute the bounds that Theorem L gives for concrete k 28 ⟩ ≡

```

fmt.Println("6. concrete bounds")
badKnown := 0
var llN [31]int
for  $k := 3; k \leq 30; k++$  {
  for  $n := k; ; n++$  {
     $D, d := \text{degrees}(k, n)$ 
    if  $ok, -, -, := \text{feasible}(\text{float64}(k), \text{float64}(D), \text{float64}(d)); \neg ok$  {
      break
    }
     $llN[k] = n$ 
  }
  if  $llN[k] \geq \text{known}[k]$  {
     $\text{badKnown}++$ 
  }
}
fmt.Printf("llN[10], llN[20], llN[30], known[10], known[20], known[30]: Theorem L gives W(3,k) > %d, %d, %d; +
the table has %d, %d, %d\n", llN[10], llN[20], llN[30], known[10],
known[20], known[30])
claim( $\text{badKnown} \equiv 0$ , "for  $3 \leq k \leq 30$  every such bound is below the table")
⟨ Compare the answer's constants with free ones for large  $k$  29 ⟩

```

This code is used in section 3.

29. For large k the degrees are the answer's $D = 3kn/(k-1)$ and $d = \frac{3}{2}kn$, and the largest n is found by bisection. Even with p , x , and y free, these n stay below $k^2/(\ln k)^3$ as far as $k = 10^8$, though the ratio grows. They do exceed $k^2/(81(\ln k)^2)$, the variant above, which itself passes $k^2/(\ln k)^3$ once $\ln k > 81$, that is, once k has 36 digits.

⟨ Compare the answer's constants with free ones for large k 29 ⟩ $\equiv$

```

var ratios []float64
badBig := 0
for e := 2; e ≤ 8; e++ {
    k := math.Pow(10, float64(e))
    if ok, -, -, - := feasible(k, 3 * k * k / (k - 1), 1.5 * k * k); ¬ok {
        fmt.Printf("UUUUUUUUUUk=10~%d: not even n=k is reached\n", e)
        continue
    }
    lo, hi := k, k * k
    for i := 0; i < 60; i++ {
        mid := math.Sqrt(lo * hi)
        if ok, -, -, - := feasible(k, 3 * k * mid / (k - 1), 1.5 * k * mid); ok {
            lo = mid
        } else {
            hi = mid
        }
    }
    L := math.Log(k)
    fmt.Printf("UUUUUUUUUUk=10~%d: n=%0.4g, which is %0.2fk^2/(ln k)^3 and " +
        "%0.3fk^2/(ln k)^2\n", e, lo, lo * L * L * L / (k * k), lo * L * L / (k * k))
    if lo < k * k / (81 * L * L) ∨ len(ratios) > 0 ∧ lo * L * L * L / (k * k) ≤ ratios[len(ratios) - 1] {
        badBig++
    }
    ratios = append(ratios, lo * L * L * L / (k * k))
}
claim (badBig ≡ 0 ∧ len(ratios) > 0 ∧ ratios[len(ratios) - 1] < 1,
    "with the answer's D and, the best n grows relative to k^2/(ln k)^3 but " +
    "stays below it, and above k^2/(81(ln k)^2)")

```

This code is used in section 28.

30. Resampling. Algorithm M on page 266 sets every x_i to 1 with probability $\xi = 1 - p$, and while some event happens it resamples the positions of one such event. Theorem M says that if the conditions of Theorem L hold with numbers θ_j , the event A_j is resampled at most $\theta_j/(1 - \theta_j)$ times on average. Here $\theta = y$ for the A 's and $\theta = x$ for the A' 's, with p, x, y from *feasible* at the largest n found for $k = 30$. The event resampled is always the first one that happens.

(Run Algorithm M 30) $\equiv$

```

fmt.Println("6. Algorithm M")
k := 30
n := k
for {
  D, d := degrees(k, n + 1)
  if ok, -, -, := feasible(float64(k), float64(D), float64(d)); !ok {
    break
  }
  n++
}
D, d := degrees(k, n)
-, p, x, y := feasible(float64(k), float64(D), float64(d))
evs := events(k, n)
r := rand.New(rand.NewPCG(seed, 30))
var countA, countB, badString int
for run := 0; run < reps; run++ {
  (Run Algorithm M once, counting resamples 31)
}
nA, nB := 0, 0
for _, e := range evs {
  if e.one {
    nB++
  } else {
    nA++
  }
}
boundA, boundB := float64(nA) * y / (1 - y), float64(nB) * x / (1 - x)
claim(badString == 0, "k=%30, n=%d, p=%.3g: %d runs, each ending with no" +
  "three equally spaced 0s and no 30 equally spaced 1s", n, p, reps)
claim(float64(countA)/float64(reps) ≤ boundA ∧ float64(countB)/float64(reps) ≤ boundB,
  "average resamples: %.3f for the %d A's (Theorem M: at most %.3f), %.3f for" +
  "the %d A' (at most %.3f)", float64(countA)/float64(reps), nA, boundA,
  float64(countB)/float64(reps), nB, boundB)

```

This code is used in section 3.

31. $\langle$ Run Algorithm M once, counting resamples 31 $\rangle \equiv$

```

bits := make([], bool, n + 1)
for i := 1; i ≤ n; i++ {
    bits[i] = r.Float64() ≥ p
}
for {
    j := -1
    for t, e := range evs {
        if allEqual(bits, e) {
            j = t
            break
        }
    }
    if j < 0 {
        break
    }
    if evs[j].one {
        countB++
    } else {
        countA++
    }
    for _, i := range evs[j].pos {
        bits[i] = r.Float64() ≥ p
    }
}

```

$\langle$ Check the final string against the definition 33 $\rangle$

This code is used in section 30.

32. $\langle$ Functions 6 $\rangle + \equiv$

```

func allEqual(bits []bool, e event) bool {
    for _, i := range e.pos {
        if bits[i] ≠ e.one {
            return false
        }
    }
    return true
}

```

33. The check does not use the events: it looks at every difference.

$\langle \text{Check the final string against the definition 33} \rangle \equiv$

```

for  $a := 1; a \leq n; a++$  {
  for  $s := 1; a + 2 * s \leq n; s++$  {
    if  $\neg bits[a] \wedge \neg bits[a + s] \wedge \neg bits[a + 2 * s]$  {
       $badString++$ 
    }
  }
}
for  $s := 1; a + (k - 1) * s \leq n; s++$  {
   $ones := \text{true}$ 
  for  $t := 0; t < k; t++$  {
     $ones = ones \wedge bits[a + t * s]$ 
  }
  if  $ones$  {
     $badString++$ 
  }
}
}

```

This code is used in section 31.

34. The solver. The clauses $waerden(3, k; n)$ say, for each three-term progression, that not all its positions are 0, and for each k -term progression, that not all are 1; here the variable x_i is true for a 1.

⟨ Functions 6 ⟩ +≡

```

func waerden(k, n int) *sat.Solver {
  s := sat.New()
  for i := 1; i ≤ n; i++ {
    s.NewVar()
  }
  for _, ap := range aps(n, 3) {
    s.AddClause(sat.Pos(ap[0]), sat.Pos(ap[1]), sat.Pos(ap[2]))
  }
  for _, ap := range aps(n, k) {
    c := make([]sat.Lit, k)
    for t, i := range ap {
      c[t] = sat.Neg(i)
    }
    s.AddClause(c...)
  }
  return s
}

```

35. For $3 \leq k \leq wmax$ the package should find a string of length $W(3, k) - 1$, which is checked against the definition, and should find none of length $W(3, k)$. For $k \leq 6$ a backtracking search that extends strings one bit at a time gives the values independently.

⟨ Recompute small values of $W(3, k)$ 35 ⟩ ≡

```

fmt.Println("6. W(3, k) with the sat package")
for k := 3; k ≤ wmax ∧ k ≤ 19; k++ {
  W := known[k]
  s := waerden(k, W - 1)
  st, _ := s.Solve(context.Background())
  good := st ≡ sat.Sat
  if good {
    ⟨ Check the model of s against the definition 37 ⟩
  }
  st2, _ := waerden(k, W).Solve(context.Background())
  claim(good ∧ st2 ≡ sat.Unsat, "W(3, %d) = %d: a checked string of length " +
    "%d, and none of length %d", k, W, W - 1, W)
}
⟨ Find  $W(3, k)$  by backtracking for  $k \leq 6$  38 ⟩

```

This code is used in section 3.

36. The table on page 189: $W(3, k)$ for $3 \leq k \leq 19$, then lower bounds for $20 \leq k \leq 30$.

⟨ Global variables 5 ⟩ +≡

```

var known = []int{3: 9, 18, 22, 32, 46, 58, 77, 97, 114, 135, 160, 186, 218,
  238, 279, 312, 349, 389, 416, 464, 516, 593, 656, 727, 770, 827, 868, 903}

```

37. $\langle \text{Check the model of } s \text{ against the definition 37} \rangle \equiv$

```

bits := make([], bool, W)
for i := 1; i < W; i++ {
    bits[i] = s.Value(sat.Pos(i))
}
for _, ap := range aps(W - 1, 3) {
    good = good ∧ (bits[ap[0]] ∨ bits[ap[1]] ∨ bits[ap[2]])
}
for _, ap := range aps(W - 1, k) {
    ones := true
    for _, i := range ap {
        ones = ones ∧ bits[i]
    }
    good = good ∧ ¬ones
}

```

This code is used in section 35.

38. The search extends a string by one bit at a time and checks only the progressions that end at the new bit; the longest string it can reach is $W(3, k) - 1$.

$\langle \text{Find } W(3, k) \text{ by backtracking for } k \leq 6 \text{ 38} \rangle \equiv$

```

for k := 3; k ≤ 6; k++ {
    bits := make([], bool, 64)
    longest := 0
    var extend func(n int)
    extend = func(n int) {
        longest = max(longest, n)
        for _, b := range []bool { false, true } {
            bits[n + 1] = b
            if ¬endsBad(bits, n + 1, k) {
                extend(n + 1)
            }
        }
    }
    extend(0)
    claim(longest + 1 ≡ known[k], "backtracking: the longest good string for " +
        "k = %d has length %d", k, longest)
}

```

This code is used in section 35.

39. Does some progression ending at position m have three 0s or k 1s?

(Functions 6) $\equiv$

```

func endsBad(bits []bool, m, k int) bool {
  for s := 1; m - 2 * s ≥ 1; s++ {
    if ¬bits[m] ∧ ¬bits[m - s] ∧ ¬bits[m - 2 * s] {
      return true
    }
  }
  for s := 1; m - (k - 1) * s ≥ 1; s++ {
    ones := true
    for t := 0; t < k; t++ {
      ones = ones ∧ bits[m - t * s]
    }
    if ones {
      return true
    }
  }
  return false
}

```

40. Index.

- a*: [8](#), [14](#), [15](#), [17](#), [22](#), [33](#).
Abs: [13](#), [24](#).
AddClause: [34](#).
allEqual: [31](#), [32](#).
alone: [15](#).
any: [6](#).
ap: [8](#), [10](#), [19](#), [20](#), [34](#), [37](#).
append: [8](#), [10](#), [19](#), [29](#).
aps: [8](#), [10](#), [19](#), [34](#), [37](#).
args: [6](#).
b: [14](#), [17](#), [38](#).
Background: [35](#).
badBig: [29](#).
badCond: [12](#), [15](#).
badD: [21](#).
badd: [21](#).
badKnown: [28](#).
badP: [13](#).
badRule: [17](#).
badString: [30](#), [33](#).
badVar: [26](#).
badYD: [24](#).
bD: [21](#).
bd: [21](#).
best: [20](#).
bits: [31](#), [32](#), [33](#), [37](#), [38](#), [39](#).
bool: [6](#), [9](#), [11](#), [16](#), [18](#), [27](#), [31](#), [32](#), [37](#), [38](#), [39](#).
both: [15](#).
boundA: [30](#).
boundB: [30](#).
c: [18](#), [20](#), [22](#), [23](#), [34](#).
claim: [6](#), [12](#), [13](#), [17](#), [21](#), [24](#), [25](#), [26](#), [28](#), [29](#), [30](#), [35](#), [38](#).
closed: [24](#).
cond: [15](#).
context: [35](#).
countA: [30](#), [31](#).
countB: [30](#), [31](#).
D: [21](#), [27](#), [28](#), [30](#).
d: [8](#), [21](#), [27](#), [28](#), [30](#).
degrees: [19](#), [21](#), [28](#), [30](#).
e: [11](#), [14](#), [16](#), [29](#), [30](#), [31](#), [32](#).
ea: [14](#), [15](#).
eb: [14](#).
ei: [18](#).
ej: [18](#).
endsBad: [38](#), [39](#).
event: [9](#), [10](#), [11](#), [16](#), [18](#), [32](#).
events: [10](#), [14](#), [17](#), [30](#).
evs: [10](#), [14](#), [17](#), [30](#), [31](#).
Exit: [7](#).
Exp: [13](#), [22](#), [23](#), [24](#), [27](#).
Expn1: [23](#).
extend: [38](#).
f: [16](#).
failures: [5](#), [6](#), [7](#).
feasible: [27](#), [28](#), [29](#), [30](#).
flag: [4](#).
Float64: [31](#).
float64: [12](#), [13](#), [14](#), [15](#), [21](#), [22](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#), [30](#).
fmt: [6](#), [7](#), [12](#), [21](#), [24](#), [28](#), [29](#), [30](#), [35](#).
format: [6](#).
from: [20](#).
good: [35](#), [37](#).
hap: [14](#), [15](#).
happens: [11](#), [14](#), [18](#).
hi: [24](#), [25](#), [29](#).
i: [11](#), [14](#), [16](#), [18](#), [19](#), [20](#), [24](#), [25](#), [26](#), [29](#), [31](#), [32](#), [34](#), [37](#).
in3: [19](#).
inK: [19](#).
int: [5](#), [8](#), [9](#), [10](#), [12](#), [17](#), [18](#), [19](#), [20](#), [21](#), [28](#), [30](#), [34](#), [36](#), [38](#), [39](#).
IntVar: [4](#).
ip: [27](#).
ix: [27](#).
j: [16](#), [19](#), [20](#), [31](#).
k: [10](#), [12](#), [13](#), [14](#), [15](#), [17](#), [19](#), [21](#), [27](#), [28](#), [29](#), [30](#), [33](#), [34](#), [35](#), [37](#), [38](#), [39](#).
known: [28](#), [35](#), [36](#), [38](#).
L: [8](#), [22](#), [23](#), [24](#), [25](#), [26](#), [29](#).
len: [18](#), [19](#), [29](#).
Lit: [34](#).
lln: [28](#).
Ln10: [24](#).
Ln2: [23](#), [24](#).
lnD: [23](#).
lnd: [23](#).
lnn: [22](#), [23](#).
lnp: [22](#), [23](#).
lnpow: [22](#), [23](#).
lnPp: [22](#), [23](#), [27](#).
lnt: [23](#).

- lnx*: [22](#), [23](#).
- lny*: [23](#), [27](#).
- lnyd*: [24](#).
- lo*: [24](#), [29](#).
- Log*: [13](#), [22](#), [23](#), [24](#), [26](#), [27](#), [29](#).
- Log1p*: [23](#), [27](#).
- long*: [19](#).
- longest*: [38](#).
- m*: [6](#), [39](#).
- main*: [3](#).
- make*: [8](#), [14](#), [19](#), [20](#), [31](#), [34](#), [37](#), [38](#).
- margin*: [22](#), [24](#), [25](#), [26](#).
- mark*: [6](#).
- math*: [13](#), [15](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [29](#).
- max*: [20](#), [21](#), [38](#).
- mid*: [24](#), [29](#).
- mode*: [4](#), [5](#), [6](#).
- most*: [19](#), [20](#).
- mustJoin*: [17](#), [18](#).
- n*: [8](#), [10](#), [12](#), [14](#), [15](#), [17](#), [18](#), [19](#), [21](#), [28](#), [30](#), [31](#), [33](#), [34](#), [38](#).
- nA*: [30](#).
- nB*: [30](#).
- nCond*: [12](#), [15](#).
- nd*: [21](#).
- Neg*: [34](#).
- New*: [12](#), [30](#), [34](#).
- NewPCG*: [12](#), [30](#).
- NewVar*: [34](#).
- ok*: [6](#), [28](#), [29](#), [30](#).
- on*: [3](#), [6](#).
- one*: [9](#), [11](#), [14](#), [15](#), [17](#), [30](#), [31](#), [32](#).
- ones*: [33](#), [37](#), [39](#).
- os*: [7](#).
- others*: [14](#), [15](#).
- out*: [8](#).
- p*: [12](#), [13](#), [14](#), [15](#), [22](#), [23](#), [27](#), [30](#), [31](#).
- pairs*: [17](#).
- Parse*: [4](#).
- Pos*: [34](#), [37](#).
- pos*: [9](#), [11](#), [16](#), [18](#), [31](#), [32](#).
- Pow*: [13](#), [15](#), [25](#), [26](#), [27](#), [29](#).
- Pp*: [13](#).
- Printf*: [6](#), [7](#), [28](#), [29](#).
- Println*: [7](#), [12](#), [21](#), [24](#), [28](#), [30](#), [35](#).
- r*: [12](#), [15](#), [30](#), [31](#).
- rand*: [12](#), [30](#).
- ratios*: [29](#).
- reps*: [4](#), [5](#), [30](#).
- run*: [30](#).
- s*: [33](#), [34](#), [35](#), [37](#), [39](#).
- Sat*: [35](#).
- sat*: [34](#), [35](#), [37](#).
- seed*: [4](#), [5](#), [12](#), [30](#).
- seen*: [20](#).
- share*: [14](#), [16](#), [17](#).
- signs*: [25](#).
- size*: [20](#).
- Solve*: [35](#).
- Solver**: [34](#).
- Sprintf*: [6](#).
- Sqrt*: [29](#).
- st*: [35](#).
- st2*: [35](#).
- string*: [5](#), [6](#).
- StringVar*: [4](#).
- sub*: [15](#).
- t*: [8](#), [18](#), [31](#), [33](#), [34](#), [39](#).
- three*: [19](#).
- through*: [20](#).
- trial*: [15](#).
- UInt64*: [15](#).
- uint64*: [5](#), [11](#), [14](#), [15](#), [18](#).
- UInt64Var*: [4](#).
- Unsat*: [35](#).
- Value*: [37](#).
- W*: [35](#), [37](#).
- w*: [14](#), [15](#).
- waarden*: [34](#), [35](#).
- want*: [17](#).
- wmax*: [4](#), [5](#), [35](#).
- worstD*: [21](#).
- worstd*: [21](#).
- x*: [11](#), [14](#), [15](#), [18](#), [19](#), [20](#), [27](#), [30](#).
- y*: [18](#), [23](#), [27](#), [30](#).

- ⟨ Check condition (133) for all events on n positions 14 ⟩ Used in section 12
- ⟨ Check that the margin changes sign once 25 ⟩ Used in section 24
- ⟨ Check the degree bounds D and d 21 ⟩ Used in section 3
- ⟨ Check the events and the lopsidedependency graph 12 ⟩ Used in section 3
- ⟨ Check the final string against the definition 33 ⟩ Used in section 31
- ⟨ Check the inequalities of Theorem L numerically 24 ⟩ Used in section 3
- ⟨ Check the model of s against the definition 37 ⟩ Used in section 35
- ⟨ Check the probabilities of the events 13 ⟩ Used in section 12
- ⟨ Check the rule of answer 351 17 ⟩ Used in section 12
- ⟨ Check the variant with $p = (3 \ln k)/k$ 26 ⟩ Used in section 24
- ⟨ Compare the answer's constants with free ones for large k 29 ⟩ Used in section 28
- ⟨ Compare the conditional probabilities of ea with its own 15 ⟩ Used in section 14
- ⟨ Compute $\ln P'$, $\ln x$, and $\ln(1 - y)^d$ 23 ⟩ Used in section 22
- ⟨ Compute the bounds that Theorem L gives for concrete k 28 ⟩ Used in section 3
- ⟨ Find $W(3, k)$ by backtracking for $k \leq 6$ 38 ⟩ Used in section 35
- ⟨ Functions 6 ⟩ Used in section 3
- ⟨ Global variables 5 ⟩ Used in section 3
- ⟨ Read the command line 4 ⟩ Used in section 3
- ⟨ Recompute small values of $W(3, k)$ 35 ⟩ Used in section 3
- ⟨ Run Algorithm M 30 ⟩ Used in section 3
- ⟨ Run Algorithm M once, counting resamples 31 ⟩ Used in section 30
- ⟨ Say how it went 7 ⟩ Used in section 3
- ⟨ Types 9 ⟩ Used in section 3

A Local Lemma bound for $W(3,k)$

	Section	Page
Introduction	1	1
Progressions and events	8	4
Events	12	5
Degrees	19	9
Numbers	22	11
Concrete bounds	27	14
Resampling	30	16
The solver	34	19
Index	40	22